군-학 연계 프로그램 범용 교육

딥러닝 기초 — 왜 딥러닝은 성공했는가

서왕덕

경기대학교 AI컴퓨터공학부 4차시

26.10.07

목차

1. 오늘의 질문
2. 1부. 딥러닝이란: 규칙 대신 예시로 배운다
3. 2부. 오래된 아이디어와 긴 겨울
4. 3부. 2012년: 세 가지 재료가 만났다
5. 4부. 왜 계속 성공하는가
6. 정리
7. 실습. AI 죄수의 딜레마

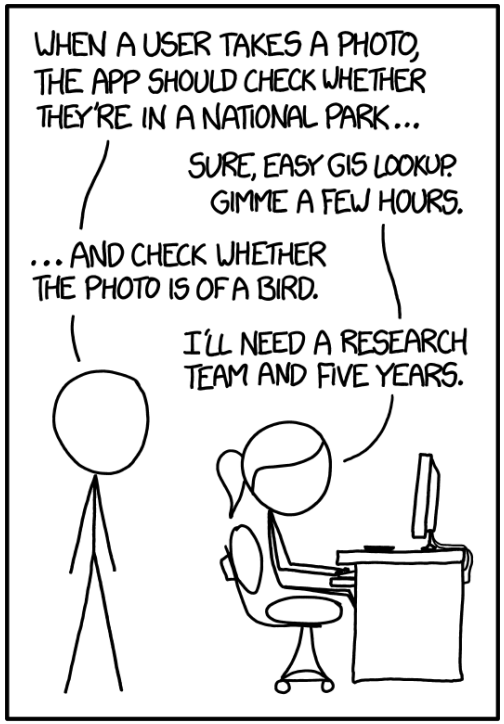
오늘 수업 로드맵 (120분)

시간	구간	내용
5분	오늘의 질문	2014년의 농담 · 왜 2012년인가
17분	**1부** 딥러닝이란	규칙 대신 예시 · 뉴런과 층 · 학습 · 스스로 배우는 특징
9분	**2부** 긴 겨울	1958년 퍼셉트론 · 두 번의 겨울 · 빠져 있던 것
17분	**3부** 2012년	이미지 인식 대회 · AlexNet · 데이터 · 계산 · 알고리즘
9분	**4부** 계속되는 성공	같은 방법 다른 문제 · 규모 · 한계 · 오늘 실습과의 연결
3분	정리	핵심 정리
60분	**실습** AI 죄수의 딜레마	팀 AI 준비 · 연습 모드 · 팀 대항전 (학생 안내문)

◎ 오늘의 목표 하나

딥러닝이 왜 2012년부터 통했는지를 **데이터 · 계산 · 알고리즘** 세 가지로 설명할 수 있다. 수식은 쓰지 않는다. 가중치는 "다이얼"이라고 부른다.

2014년의 농담



xkcd 1425 "Tasks" (2014.9.24)
CC BY-NC 2.5

만화의 대화

기획자: 사진을 찍으면 국립공원 안인지 확인해 주세요.
개발자: 위치 정보만 조회하면 됩니다. 몇 시간이면 돼요.
기획자: 그리고 새를 찍은 사진인지도 확인해 주세요.
개발자: 연구팀과 5년이 필요합니다.

(아래 글) 컴퓨터 분야에서는 쉬운 일과 사실상 불가능한 일의 차이를 설명하기 어렵다.

한 달 뒤

2014년 10월, 사진 공유 사이트 Flickr가 사진 속 새를 가려내는 "Park or Bird"를 공개했다. **딥러닝**으로 만들었다.

❓ 사람에게 쉬운 "새인지 보기"는 컴퓨터에게 약 50년 동안 풀리지 않던 문제였다. 지금은 휴대폰 사진첩이 "새"를 검색해 준다. 그 사이에 무슨 일이 있었나.

오늘의 질문

질문

신경망이라는 아이디어는 **1958년**에 나왔다.
그런데 왜 **2012년**이 되어서야 통했는가?



1958년

글자를 배우는 기계 "퍼셉트론"이 공개됐다. 신문은 곧 걷고 말하는 기계가 나온다고 썼다



그 뒤 50년

두 번의 침체기. 신경망 연구는 연구비도 관심도 잃었다



2012년

이미지 인식 대회에서 신경망이 2위와 10%p 넘는 차이로 1위를 했다

답을 먼저 말하면

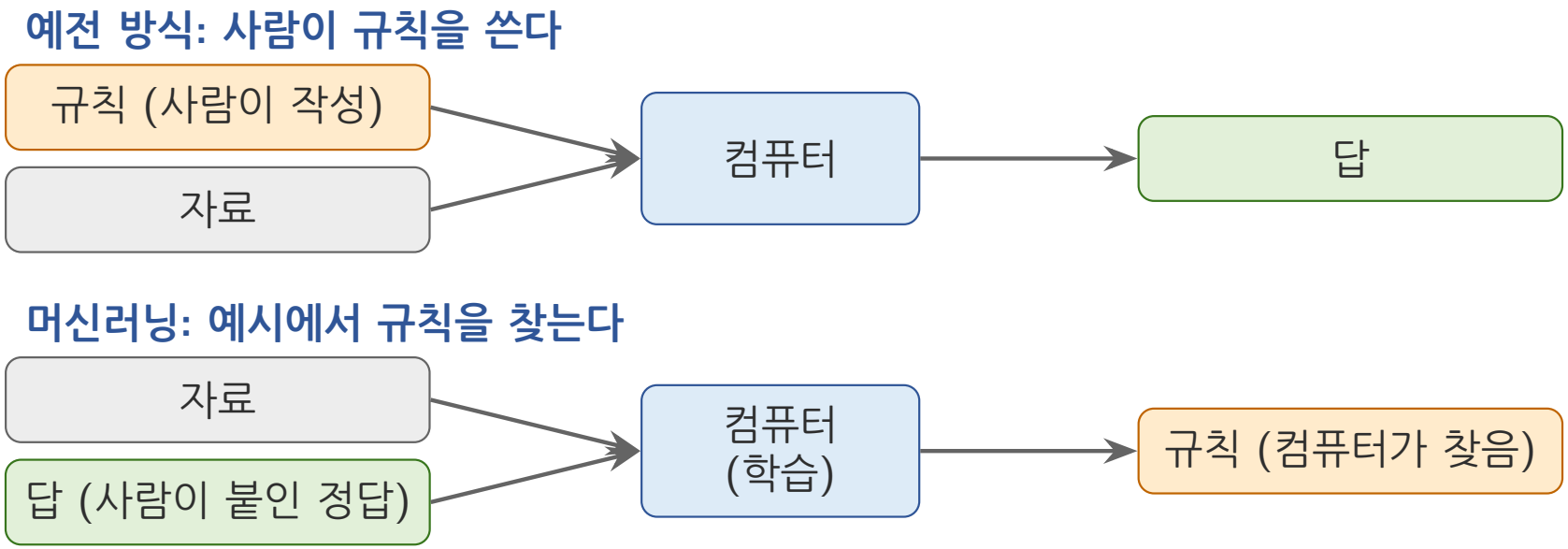
아이디어가 틀렸던 것이 아니다. **데이터 · 계산 · 알고리즘** 세 가지가 모자랐고, 2012년 무렵 한꺼번에 갖춰졌다. 오늘 60분은 이 문장을 풀어 쓰는 시간이다.



1부 딥러닝이란

규칙 대신 예시로 배운다

규칙을 쓰는 방법과 예시로 배우는 방법



스팸 메일을 규칙으로

"무료"가 세 번 넘게 나오면 스팸. 새 수법이 나올 때마다 사람이 규칙을 고친다.

스팸 메일을 예시로

스팸 1만 통과 정상 메일 1만 통을 보여 주고 차이를 찾게 한다. 새 수법은 새 예시로 배운다.

예시로 배우는 방법을 통틀어 **머신러닝**이라 하고, 그중 **층을 깊게 쌓은 신경망**으로 배우는 것이 **딥러닝**이다.

생각해 보기: 고양이를 규칙으로 쓴다면



활동 (2분)

여섯 장이 모두 고양이임을 가려내는 **규칙 세 줄**을 옆 사람과 써 보세요.

써 보면

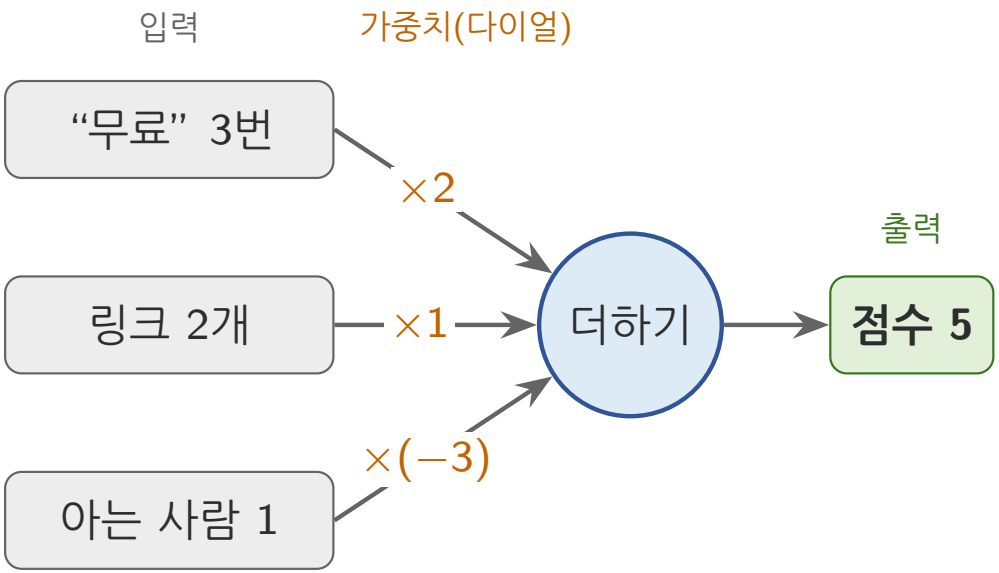
귀가 뾰족하다 → 뒷모습은?
수염이 있다 → 어두운 사진은?
다리가 넷이다 → 웅크리면?

사람은 한눈에 알지만 규칙으로는 못 쓴다

1966년 MIT는 "카메라에 비친 물체 알아보기"를 여름 한 철 과제로 냈다. 풀리지 않았다. 2010년에도 가장 좋은 방법의 오류율이 28%였다(3부에서 다시 본다).

사진(윗줄 왼쪽부터): Eyyaahh (CC BY-SA 3.0), Matthew Paul Argall (CC0), Luke Rogers (CC BY-SA 4.0), Kevin Dooley (CC BY 2.0), Luke Rogers (CC BY-SA 4.0), AlmightyWorm (CC0). Wikimedia Commons

인공 뉴런 하나가 하는 일



뉴런이 하는 일

입력마다 **가중치**를 곱해서 더한다. 가중치는 그 입력을 얼마나 중요하게 볼지 정하는 **다이얼**이다.

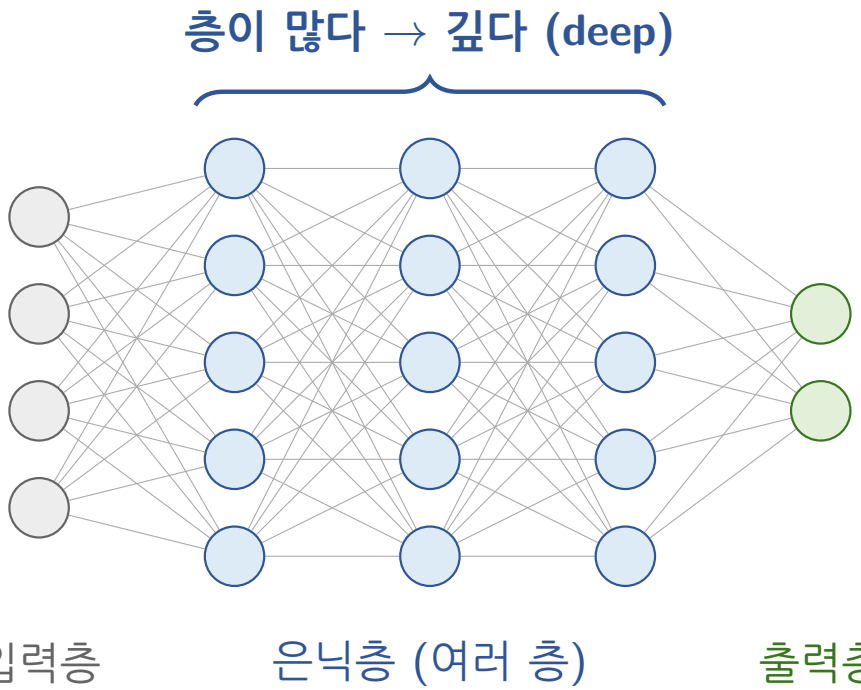
계산해 보면

$3 \times 2 + 2 \times 1 + 1 \times (-3) = 5$
기준(예: 4)을 넘으면 “스팸”.

학습이란

다이얼 값을 사람이 정하지 않는다. 예시를 보고 맞는 값을 **찾는다**. 딥러닝에서 “학습”은 이 다이얼 맞추기를 가리킨다. 뉴런 하나는 곱하기와 더하기뿐이다.

층을 쌓으면 신경망이 된다



딥러닝

뉴런을 층으로 묶고 층을 여러 개 쌓는다. 앞 층의 출력이 다음 층의 입력이 된다. **선 하나가 다이얼 하나**다.

다이얼의 수

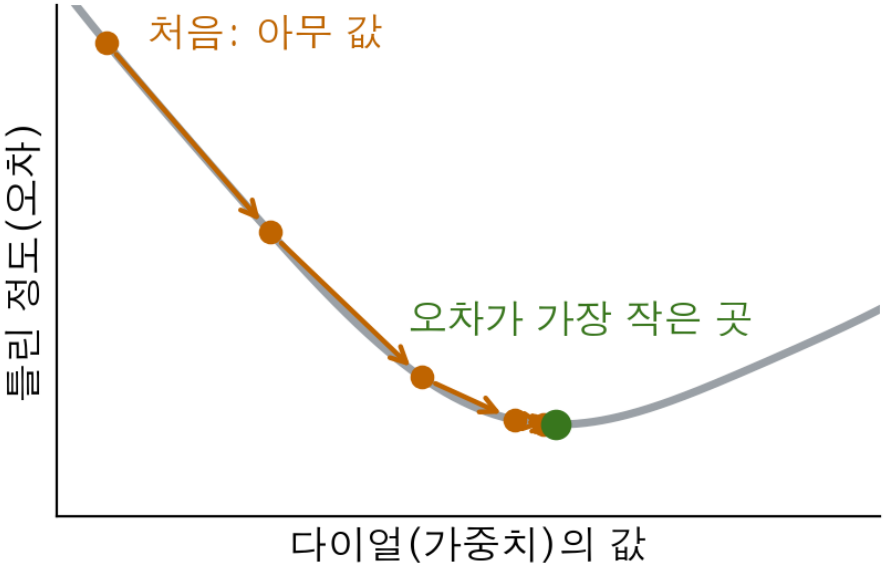
1998	LeNet-5	6만 개
2012	AlexNet	6천만 개
2020	GPT-3	1,750억 개

구조는 단순하다. 곱하고 더하는 뉴런을 많이 이어 붙였을 뿐이다. 어려운 것은 이 많은 다이얼을 **어떻게 맞추느냐**다.

학습: 틀린 만큼 다이얼을 조금씩 돌린다

학습의 한 바퀴

1) 예시 하나를 넣는다 (사진과 정답)
2) 답을 정답과 비교한다. 차이가 **오차**
3) 오차가 줄어드는 쪽으로 다이얼을 돌린다
4) 다음 예시로. 수백만 번 되풀이한다



다이얼 하나만 그린 개념도 (실제로는 수천만 개)

비유

안개 낀 산에서 발밑 경사만 보고 한 걸음씩 내려가는 것과 같다.

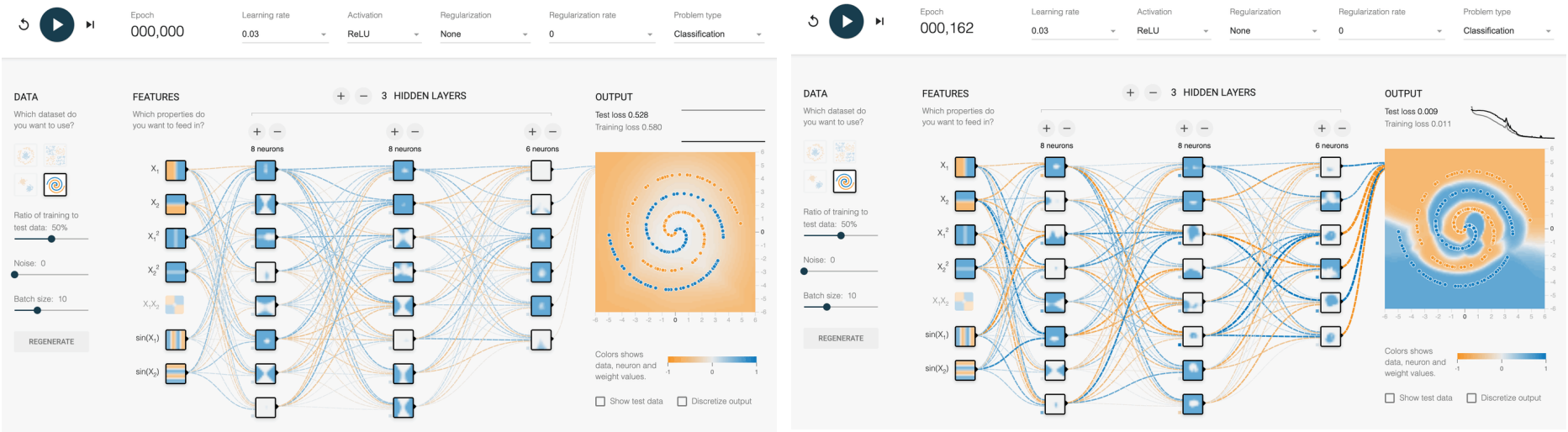
역전파 (1986)

출력의 오차를 뒤에서 앞으로 전달해, 모든 다이얼이 어느 쪽으로 얼마나 움직여야 하는지를 한 번에 계산하는 방법. 지금의 딥러닝도 이 방법으로 배운다.

눈으로 보기: 신경망이 배우는 과정

학습 전 (0회): 시험 오차 0.528

학습 162회 뒤: 시험 오차 0.009



TensorFlow Playground 화면 캡처 (Apache 2.0). 왼쪽부터 입력, 은닉층 3개(뉴런 8 · 8 · 6개), 출력

무엇을 보는가

주황 점과 파랑 점이 나선으로 섞여 있다. 신경망은 두 색을 가르는 경계를 배운다. 출력 그림의 **배경색**이 신경망의 답이다.

직접 해 보기

`playground.tensorflow.org`
▶ 버튼만 누르면 된다. 층과 뉴런을 줄이면 경계가 단순해진다.

층마다 배우는 것이 다르다



선 · 모서리 → 질감 → 무늬 → 부분 → 물체 (앞쪽 층 → 뒤쪽 층)

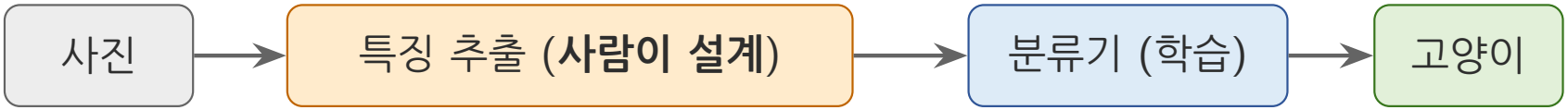
이미지 인식 신경망의 각 층 뉴런이 가장 강하게 반응하는 그림. Olah et al., "Feature Visualization", Distill (2017), CC BY 4.0, 라벨 크기 조정

단계는 학습 중에 저절로 생겼다

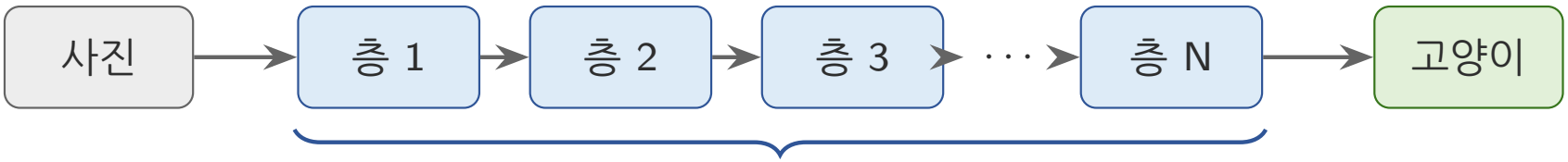
앞쪽 층은 선과 모서리에, 가운데 층은 질감과 무늬에, 뒤쪽 층은 눈 · 바퀴 같은 부분과 물체 전체에 반응한다. 누구도 "선부터 찾아라"라고 시키지 않았다. 준 것은 사진과 정답뿐이다.

핵심: 특징을 누가 만드는가

2012년 이전



딥러닝



1부의 결론

딥러닝은 **특징 설계를 사람 손에서 데이터로** 넘겼다. 사람의 설계가 한계가 아니게 되므로, 데이터와 계산을 늘리면 성능이 따라 오른다. 이 성질이 3부와 4부의 출발점이다.

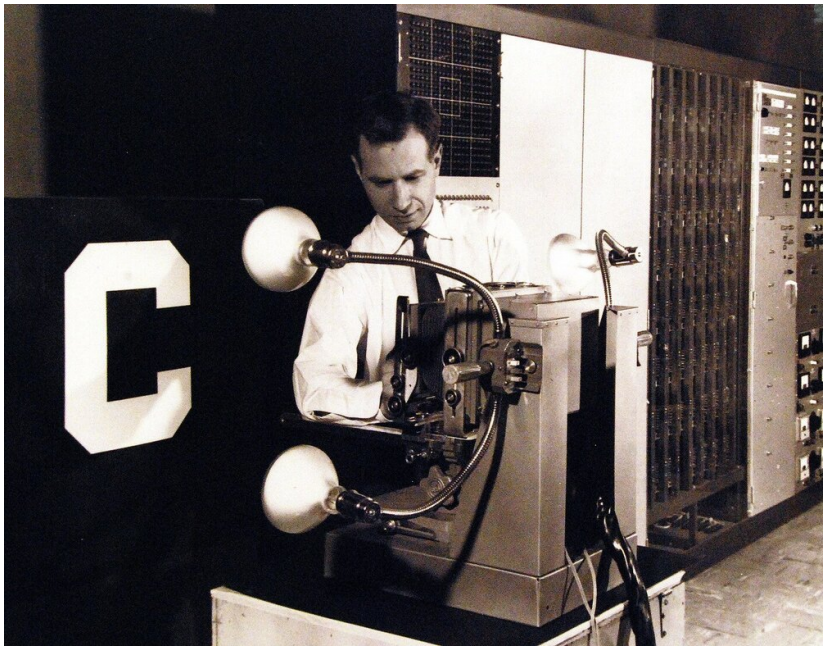
예전 방식의 특징은 "모서리 방향의 분포" 같은 계산식이었다. 연구자들이 문제마다 몇 년씩 다듬었다.



2부 긴 겨울

오래된 아이디어는 왜 50년을 기다렸나

1958년: 글자를 배우는 기계



Mark I 퍼셉트론 (1960년 공개 사진)
U.S. Navy, Public domain

퍼셉트론 (Frank Rosenblatt)

1957년 고안. 광센서 400개(20×20)로 글자 모양을 받아, 가중치를 스스로 조정하며 배웠다. 1부에서 본 "뉴런 하나"와 같은 구조다.

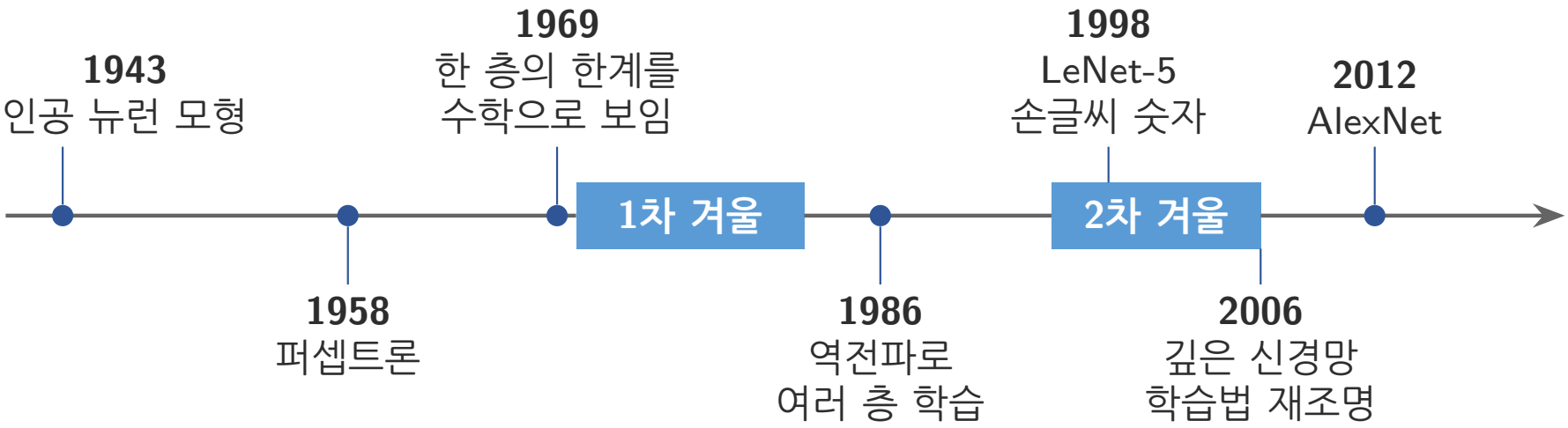
뉴욕타임스 (1958.7.8)

"걷고, 말하고, 보고, 쓰고, 스스로를 복제하고, 자신의 존재를 의식하게 될 전자 컴퓨터의 배아"

실제

층이 하나뿐이라 풀 수 있는 문제가 좁았다. 기대와 실제의 차이가 첫 번째 겨울로 이어졌다.

두 번의 겨울



1943 McCulloch · Pitts / 1958 Rosenblatt / 1969 Minsky · Papert / 1986 Rumelhart · Hinton · Williams
1998 LeCun 외 / 2006 Hinton 외 / 겨울의 시기는 대략

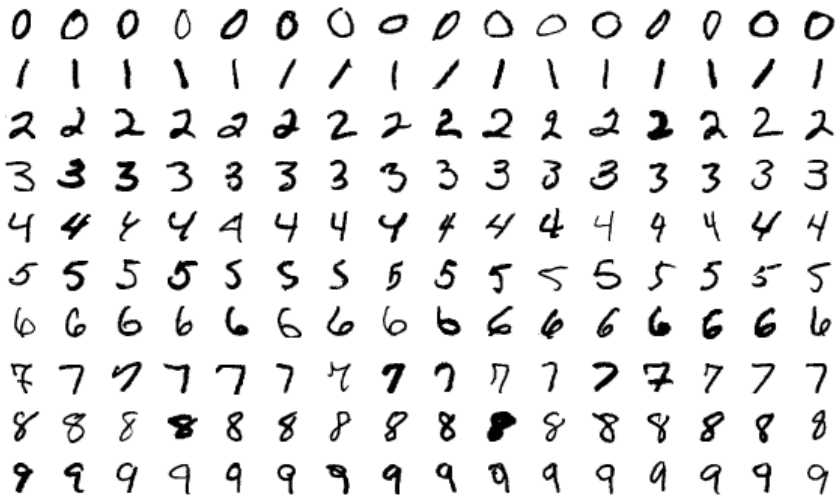
❄ 겨울의 공통점

큰 기대, 실제 문제에서 드러난 한계, 연구비와 관심의 감소. 아이디어가 틀려서가 아니라 조건이 모자랐다.

그동안 쓰인 방법

사람이 특징을 설계하고 분류기만 학습시키는 방식(1부의 "2012년 이전")이 2000년대의 표준이었다.

그래도 되는 것은 있었다: 손글씨 숫자



손글씨 숫자 자료 MNIST의 예. 그림: Josef Steppan, CC BY-SA 4.0

LeNet (Yann LeCun 연구진)

1989년 손글씨 우편번호를 읽었고 1998년 LeNet-5로 이어졌다. 다이얼은 약 6만 개.

실제로 쓰였다

1996년부터 은행 수표 판독기에 들어갔다. 2001년 기준 하루 약 2천만 장, 미국 수표의 약 10%를 읽었다.

그러나

작은 흑백 숫자 10종까지였다. 컬러 사진 속 물체 1,000종으로 키울 데이터도 컴퓨터도 없었다.

무엇이 빠져 있었나

구조는 이미 있었다. 모자란 것은 세 가지였다.



데이터

정답이 붙은 사진이 수만 장 규모였다. 다이얼 수천만 개를 맞추기에는 예시가 모자랐다



계산

큰 신경망을 학습시키기에는 당시 컴퓨터가 수십 배 느렸다



알고리즘

층을 깊게 쌓으면 오차 신호가 앞쪽 층까지 전달되지 않아 학습이 멈췄다

셋은 따로 풀리지 않는다

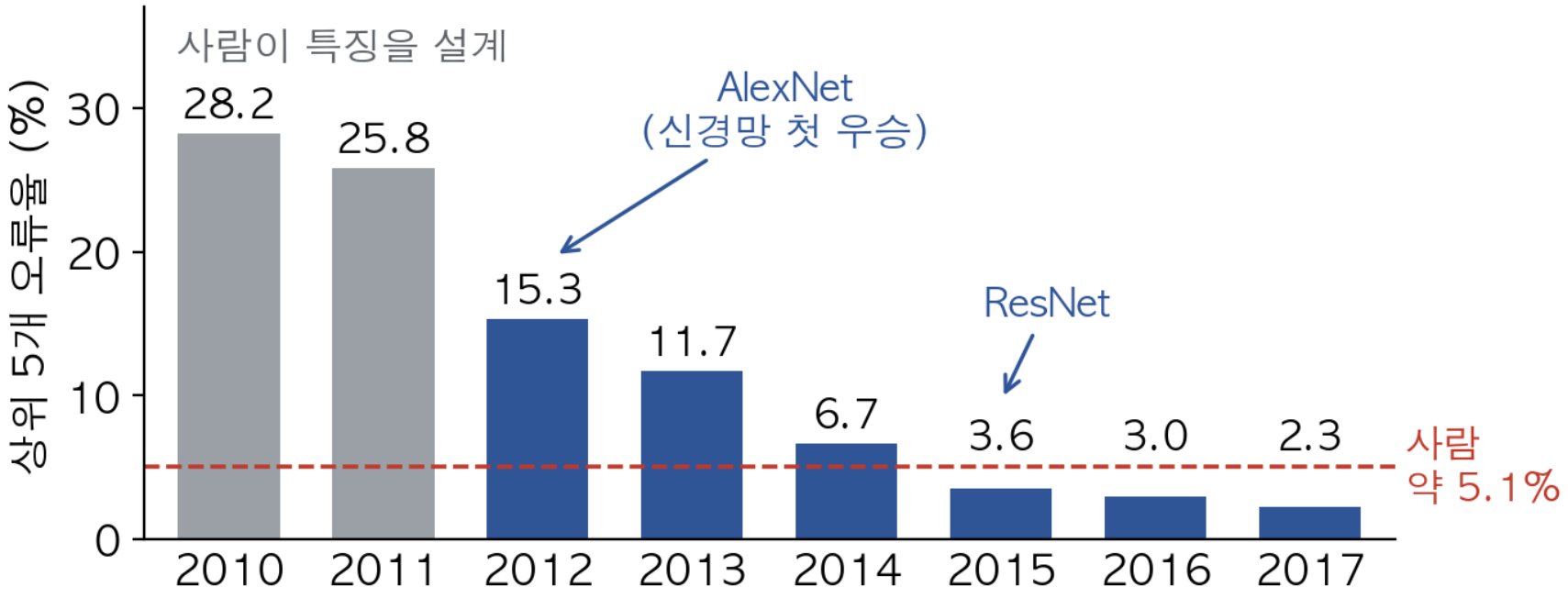
데이터가 많아도 계산이 느리면 쓰지 못한다. 계산이 빨라도 깊은 신경망이 학습되지 않으면 소용없다. 2012년 무렵 셋이 함께 풀렸다. 3부는 그 이야기다.



3부 2012년

세 가지 재료가 만났다

이미지 인식 대회에서 일어난 일



ImageNet 대회(ILSVRC) 연도별 1위의 오류율. 자료: 대회 공식 결과, Russakovsky et al. (2015)

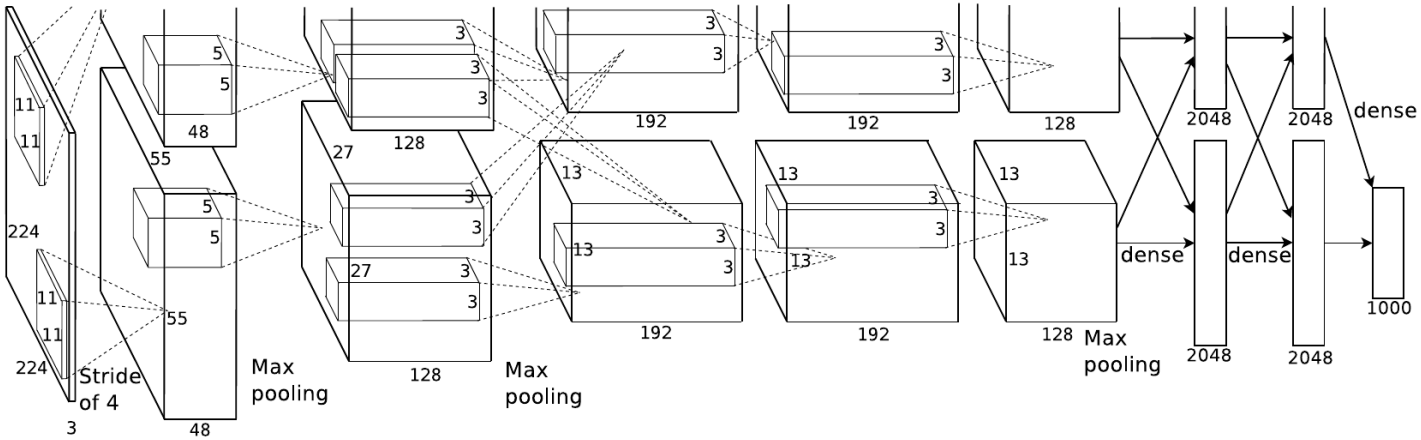
대회 규칙

사진 한 장에 답을 5개 낸다. 그 안에 정답이 없으면 오답. 물체 1,000종, 시험 사진 10만 장.

2012년

1위 15.3%, 2위 26.2%. 1위만 신경망이었다. 이듬해 상위권은 대부분 신경망으로 바뀌었다.

AlexNet (2012)



Krizhevsky, Sutskever, Hinton, "ImageNet Classification with Deep Convolutional Neural Networks" (2012) 그림 2

8층
다이얼이 있는 층

6천만 개
다이얼(가중치)

120만 장
학습에 쓴 사진

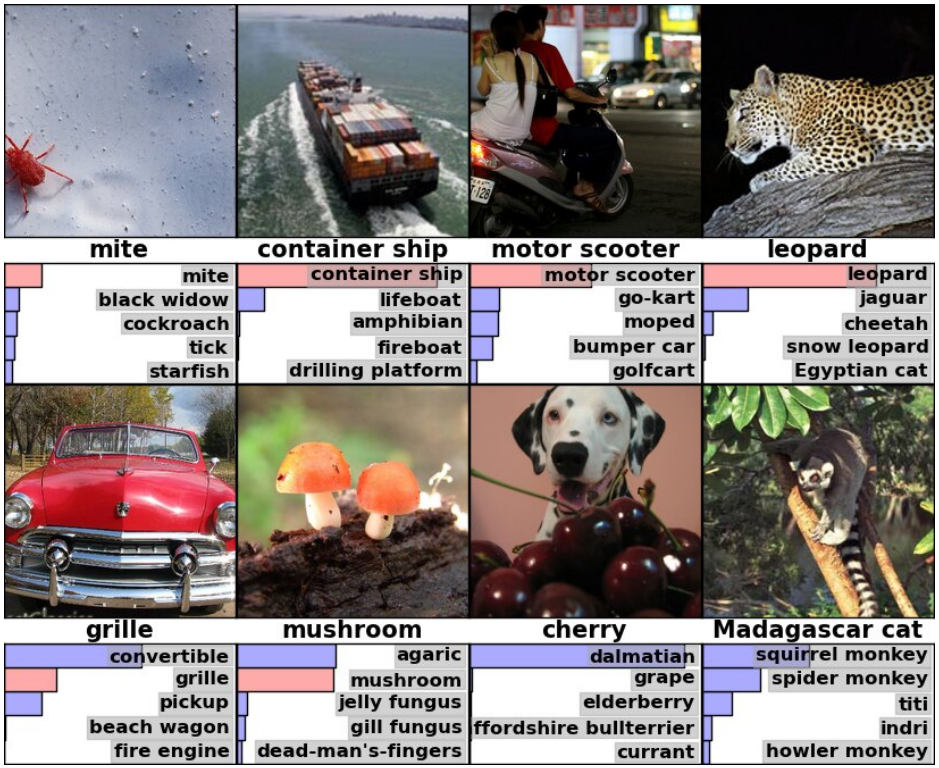
GPU 2장
그래픽카드

5~6일
학습에 걸린 시간

새로 발명된 구조가 아니다

토론토대 대학원생 Alex Krizhevsky가 지도교수 Hinton, 동료 Sutskever와 만들었다. 구조는 1998년 LeNet과 같은 계열이다. 달라진 것은 **크기 · 데이터 · 계산**이다.

AlexNet이 낸 답



같은 논문 그림 4의 일부

읽는 법

사진 바로 아래 글씨가 정답. 막대 다섯 개는 신경망이 고른 답과 확신의 크기. 분홍 막대가 정답과 같은 답이다.

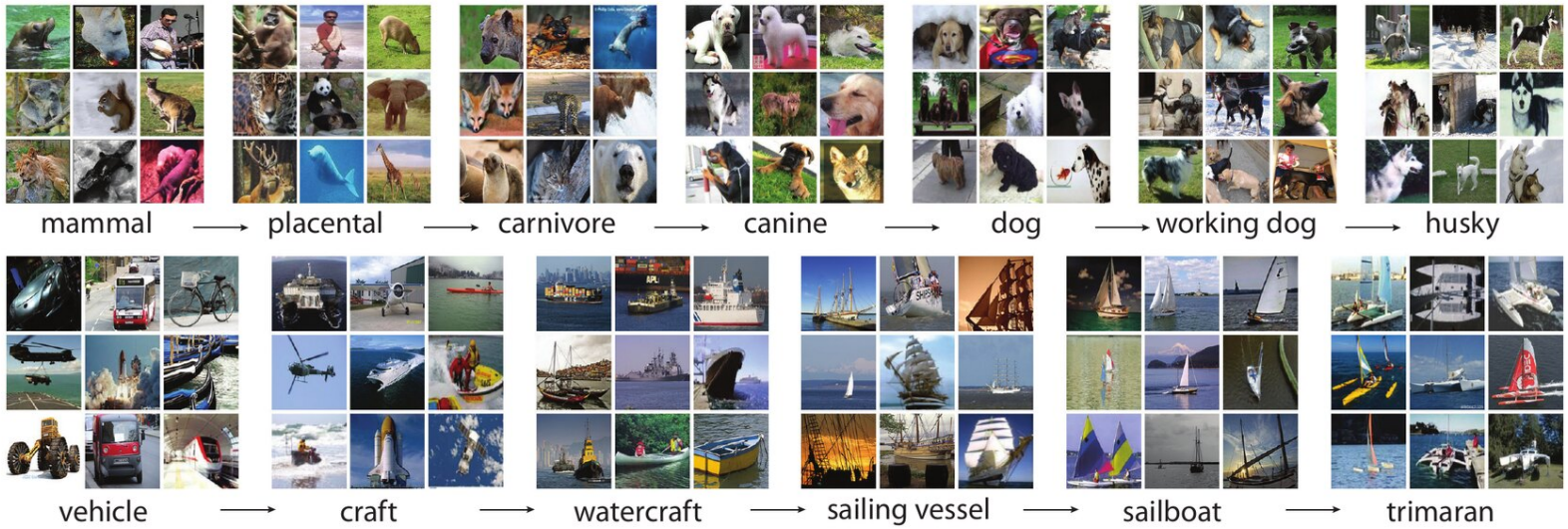
윗줄

진드기 · 컨테이너선 · 스쿠터 · 표범. 네 장 모두 첫 번째 답이 정답이다.

아랫줄 세 번째

정답은 "체리"인데 "달마시안"이라고 답했다. 사진에 달마시안이 실제로 있다. 틀렸지만 엉뚱한 답은 아니다.

재료 1 데이터: ImageNet



Deng et al., "ImageNet: A Large-Scale Hierarchical Image Database" (CVPR 2009) 그림 1



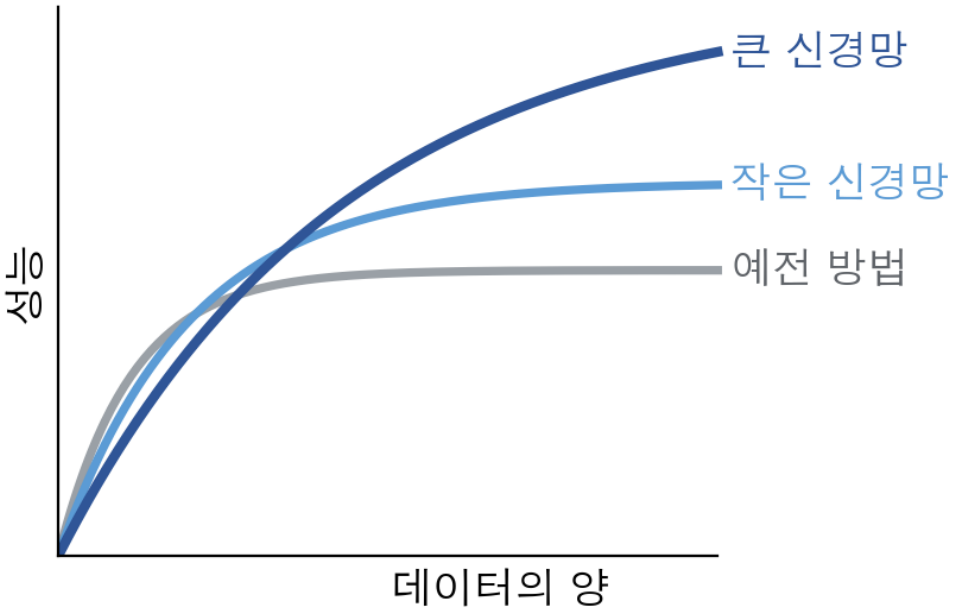
Fei-Fei Li

ImageNet (2009년 공개, Fei-Fei Li 연구진)

사진 약 1,420만 장에 사람이 이름표를 붙였다. 범주는 21,841개. 2008년 7월부터 2010년 4월까지 167개국 4만 9천 명이 온라인으로 참여했다. 대회에는 그중 1,000종 128만 장을 학습용으로 썼다. 그 전까지 널리 쓰이던 자료는 수만 장 규모였다.

인물 사진: ITU Pictures, CC BY 2.0

데이터가 많을수록 큰 신경망이 유리하다



Andrew Ng의 설명 도식을 따라 그린 개념도 (실측값 아님)

그림이 말하는 것

예전 방법은 데이터가 늘어도 일찍 멈춘다. 큰 신경망은 데이터가 늘수록 계속 좋아진다.

데이터는 어디서 왔나

디지털카메라와 스마트폰, 사진 공유 사이트, 검색 엔진. 이름표는 온라인으로 모은 사람들이 붙였다.

왼쪽 끝을 보면

데이터가 적을 때는 신경망이 유리하지 않다. 2000년대까지 신경망이 밀렸던 이유 가운데 하나다.

재료 2 계산: 게임용 그래픽카드



GeForce GTX 580 (2010년 출시). 사진: TheStriker, CC BY-SA 4.0

CPU와 GPU

CPU는 복잡한 일을 차례로 처리한다. GPU는 단순한 곱셈과 덧셈 수천 개를 **동시에** 처리한다.

왜 그래픽카드인가

화면의 점 수백만 개를 동시에 계산하려고 만든 부품이다. 신경망 계산도 같은 모양이다. 곱하고 더하기를 아주 많이, 동시에 한다.

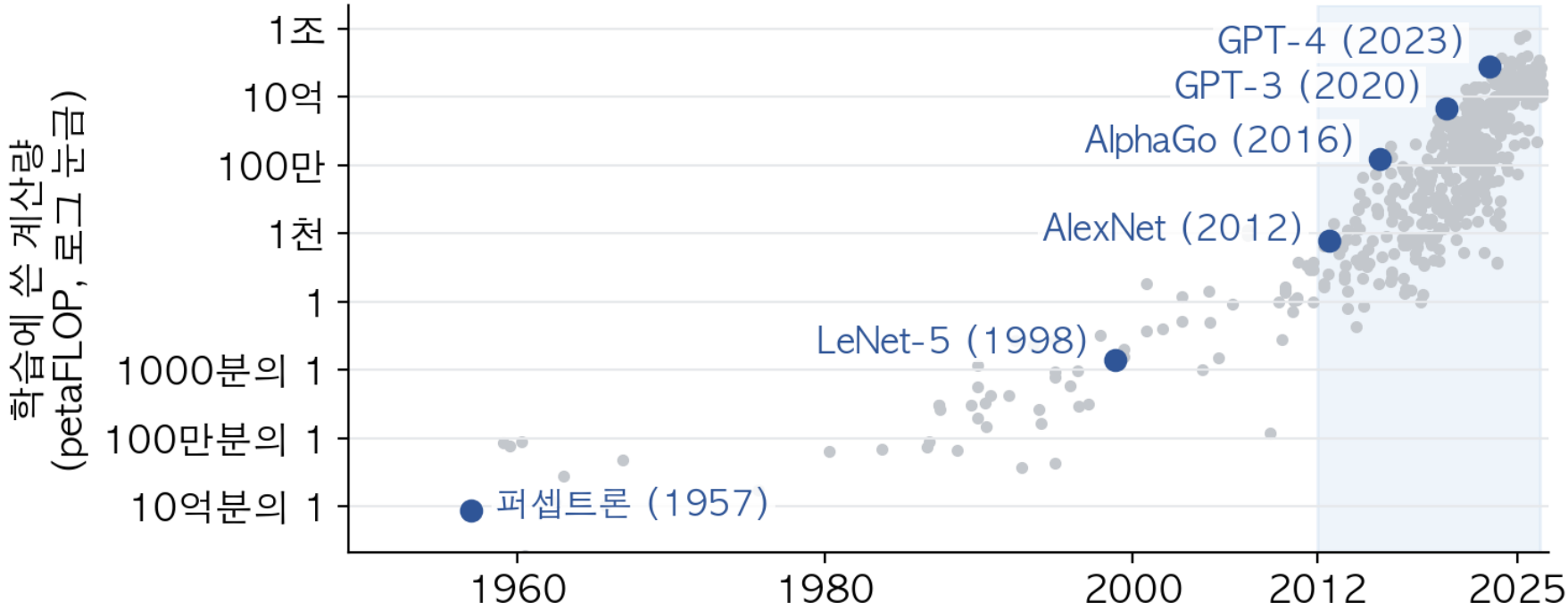
AlexNet의 경우

GTX 580 두 장으로 5~6일. 카드 한 장의 메모리(3GB)에 다 들어가지 않아 신경망을 둘로 나눠 올렸다.

오늘 실습

Colab에서 고르는 "T4"도 GPU다. 켜지 않으면 AI의 답이 훨씬 느려진다.

계산량은 얼마나 늘었나



점 하나가 AI 시스템 하나. 1 petaFLOP은 계산 10^{15}번. 자료: Epoch AI (2026), Our World in Data, CC BY

2012년에서 2023년까지

AlexNet 470 → GPT-4 약 210억.
단위는 petaFLOP.
11년 사이 약 4,500만 배.

늘어나는 속도

2010년 이전에는 약 20개월마다 2배, 그 뒤로는 약 6개월마다 2배 (Sevilla 외, 2022).

재료 3 알고리즘: 깊어도 학습되게

ReLU
AlexNet이 사용
뉴런의 출력 방식을 단순하게 바꿨다. 오차 신호가 앞쪽 층까지 전달된다

드롭아웃
2012년
학습 중 뉴런 일부를 무작위로 끈다. 예시를 통째로 외우는 것을 막는다

잔차 연결
2015년
층을 건너뛰는 지름길을 둔다. 152층까지 학습됐다 (ResNet)

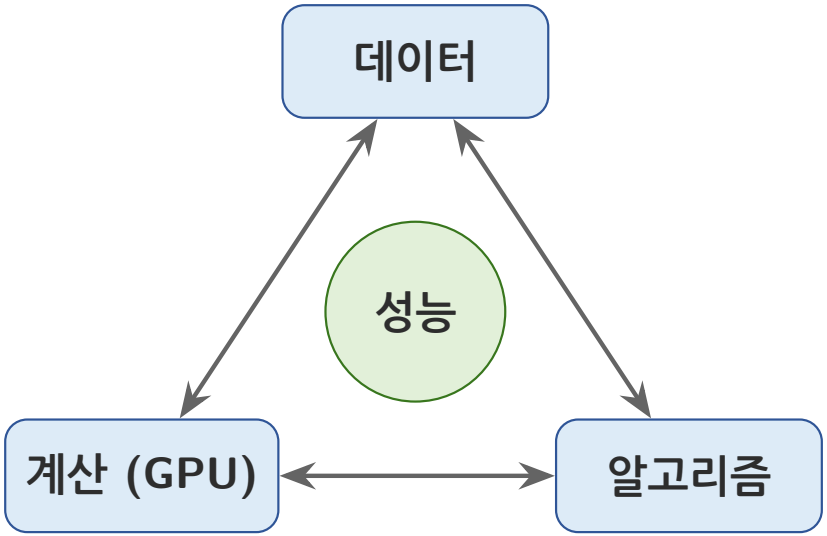
Transformer
2017년
문장 속 단어들이 서로를 직접 참조한다. 지금 언어 모델의 뼈대다

공통점

새 이론이라기보다 "큰 신경망이 실제로 학습되게 하는 요령"이다. 데이터와 GPU가 있어야 시험해 볼 수 있었으므로 2012년 이후에 빠르게 쌓였다.

뼈대인 역전파(1986)와 합성곱 신경망(1989)은 이미 있었다. AlexNet은 ReLU와 드롭아웃을 함께 썼다.

세 재료는 서로를 키운다



맞물려 돈다

데이터가 많아야 큰 모델이 의미 있고, GPU가 있어야 큰 모델을 돌리며, 학습 요령이 있어야 깊은 모델이 배운다. 성능이 오르면 투자와 사용자가 늘어 데이터와 GPU가 다시 늘어난다.

네 번째 재료: 공개

논문은 바로 공개하고, 코드와 도구도 공개했다(TensorFlow 2015, PyTorch 2016). 한 팀의 요령이 몇 달 안에 모두의 출발점이 됐다.

3부의 결론

2012년은 한 사람의 발명이 아니다. 50년 된 아이디어 위에 데이터 · 계산 · 알고리즘이 **동시에** 갖춰진 시점이다.



4부 계속되는 성공

같은 방법이 다른 문제를 푼다

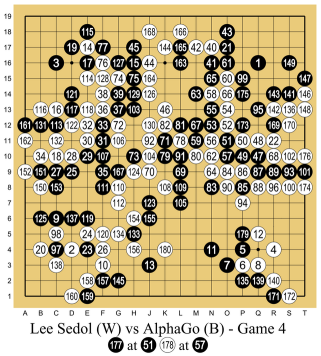
같은 방법, 다른 문제

2012 이미지
AlexNet. 사진 속 물체 1,000종 분류

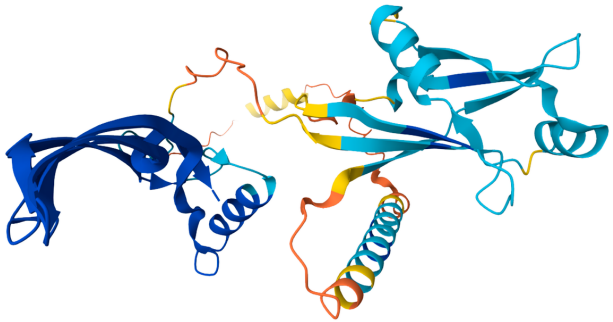
2016 바둑
AlphaGo. 서울에서 이세돌 9단에게 4승 1패

2020 단백질
AlphaFold 2. 아미노산 서열에서 입체 구조를 예측

2022 언어
ChatGPT 공개. 글을 읽고 쓰는 모델



2016년 4국 기보



AlphaFold가 예측한 단백질 구조

그대로인 것

문제마다 사람이 따로 설계하던 부분이 사라졌다. 바뀐 것은 데이터와 신경망의 모양이고, "예시로 다이얼을 맞춘다"는 그대로다.

기보: Wesalius, CC BY-SA 4.0. 단백질 구조: AlphaFold DB (AF-Q99988-F1), CC BY 4.0

쓴 교훈 (The Bitter Lesson)

Rich Sutton, 2019

"70년의 AI 연구에서 읽을 수 있는 가장 큰 교훈은, **계산을 활용하는 일반적인 방법**이 결국 가장 효과적이며 그 차이도 크다는 것이다."

사람의 지식을 넣는 방법

체스의 정석, 음성의 발음 규칙, 이미지의 특징 설계. 처음에는 잘 되지만 곧 한계에 닿는다.

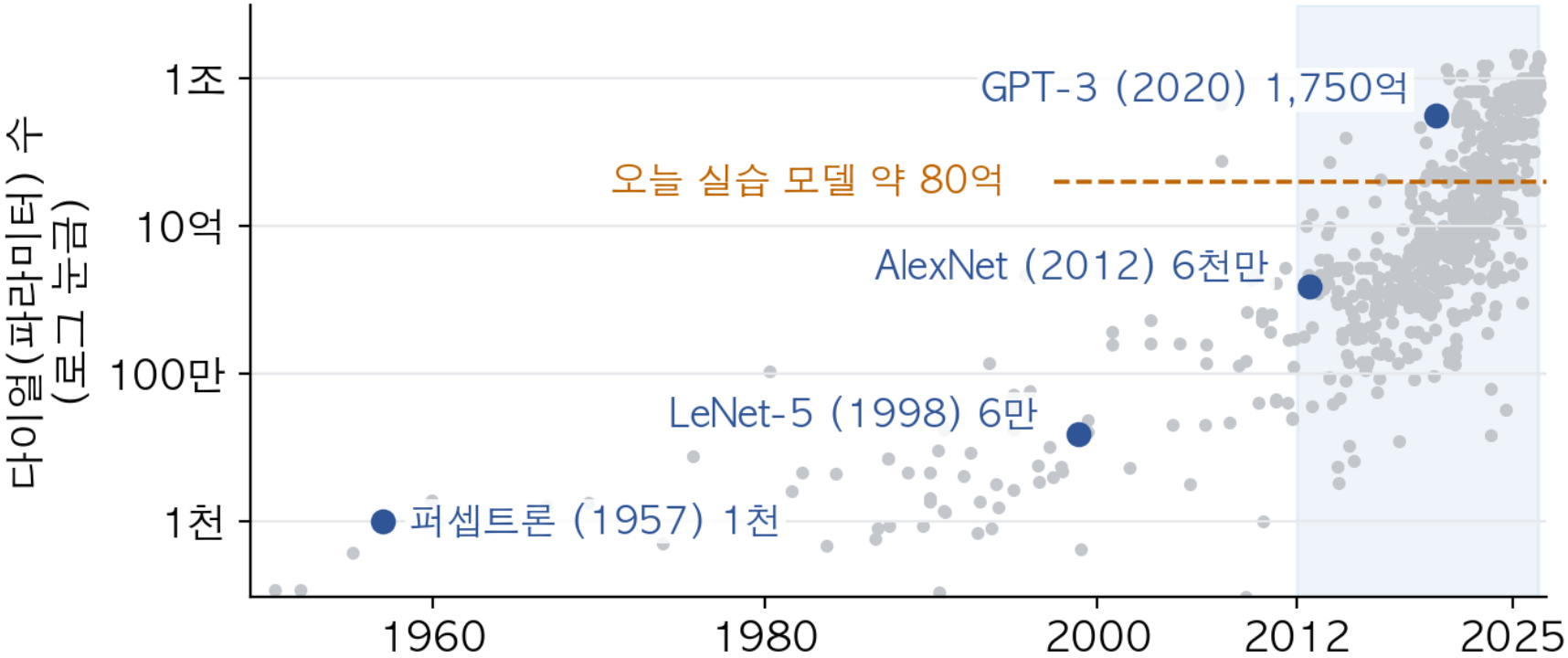
계산을 활용하는 방법

탐색과 학습. 컴퓨터가 빨라지고 데이터가 늘수록 계속 좋아진다.

왜 "쓴" 교훈인가

연구자가 공들여 넣은 지식보다, 단순한 방법에 계산을 더한 쪽이 번번이 이겼기 때문이다. 딥러닝이 계속 성공하는 이유를 한 문장으로 줄이면 이것이다. 1부의 결론(특징을 데이터에 맡긴다)과 같은 말이다.

규모: 다이얼은 얼마나 늘었나



점 하나가 AI 시스템 하나. 자료: Epoch AI (2026), Our World in Data, CC BY. 실습 모델 수치는 Ollama 모델 정보

60여 년

1957년 1천 개 → 1998년 6만 → 2012년 6천만 → 2020년 1,750억 개.

오늘 실습 모델

gemma4:e4b는 약 80억 개. AlexNet의 약 130배가 무료 Colab에서 돌아간다.

학계의 평가 1: 2018년 튜링상

Fathers of the Deep Learning Revolution Receive ACM A.M. Turing Award

Bengio, Hinton and LeCun Ushered in Major Breakthroughs in Artificial Intelligence

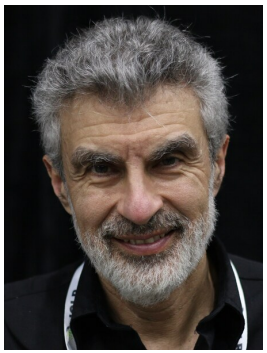
ACM 보도자료 (2019.3) 화면



Geoffrey Hinton



Yann LeCun



Yoshua Bengio

튜링상

컴퓨터 과학 분야에서 가장 권위 있는 상이다. 수상 이유는 "심층 신경망을 컴퓨팅의 핵심 요소로 만든 돌파구".

세 사람의 공통점

2부의 겨울 동안에도 신경망 연구를 이어 왔다. 역전파(Hinton), LeNet(LeCun)이 이들의 일이다.

사진: Arthur Petron (CC BY-SA 4.0), Jérémy Barande (CC BY-SA 2.0), Xuthoria (CC BY-SA 4.0)

학계의 평가 2: 2024년 노벨상

8 October 2024

The Royal Swedish Academy of Sciences has decided to award the Nobel Prize in Physics 2024 to

John J. Hopfield
Princeton University, NJ, USA

Geoffrey Hinton
University of Toronto, Canada

"for foundational discoveries and inventions that enable machine learning with artificial neural networks"

노벨위원회 보도자료 (2024.10.8) 화면

물리학상

Hopfield · Hinton. 선정 이유는 "인공 신경망으로 기계 학습을 가능하게 한 기초 발견과 발명".

화학상

Hassabis · Jumper는 단백질 구조를 예측하는 AlphaFold로, Baker는 단백질 설계로 받았다.

한 해에 두 개

과학 분야 노벨상 셋 가운데 둘이 신경망과 이어진 연구에 주어졌다. 2부의 "겨울"과 견주어 볼 만하다.

그래도 남는 한계

데이터와 비용
많은 예시와 GPU, 전력이 든다. 가장 큰 모델의 학습은 소수의 기업만 할 수 있다

설명의 어려움
다이얼 수십억 개가 왜 그 답을 냈는지 사람이 읽어 내기 어렵다

그럴듯한 오답
모르는 것도 자신 있게 답한다. 1차시와 3차시에서 다룬 문제다

치우친 예시
예시가 치우쳐 있으면 답도 치우친다. 배운 범위 밖에서는 약하다

성공의 이유와 한계의 이유는 같다

규칙이 아니라 예시에서 배웠기 때문에 잘하고, 예시에서 배웠기 때문에 틀린다. 그래서 1차시의 “구체적으로 지시하기”와 3차시의 “근거 문서를 주기”가 필요했다.

오늘 실습과의 연결

실습에서 팀 대신 협상하는 AI(gemma4:e4b)에 세 재료가 그대로 들어 있다.



데이터

대량의 글로 이미 학습을 마친 모델이다. 우리는 학습시키지 않고 내려받아 쓴다



계산

Colab의 T4 GPU에 모델을 올린다. "AI 준비" 셀이 하는 일이 이것이다



알고리즘

42층 Transformer. 다이얼 약 80억 개가 6.6GB 파일에 들어 있다

실습에서 볼 것

전략 프롬프트를 주면 AI가 생각한 뒤 협상하고 버튼을 고른다. 답 한 번에 약 5초. 그동안 GPU는 단어 조각 하나를 낼 때마다 수십억 개의 다이얼을 한 번씩 계산한다.

전략을 쓸 때

예시에서 배운 모델이므로 애매한 지시에는 애매하게 답한다. 조건과 태도를 구체적으로 쓴다 (1차시).

오늘의 핵심 정리

왜 딥러닝은 성공했는가

1. **규칙 대신 예시로 배운다.** 층을 깊게 쌓은 신경망의 다이얼(가중치)을, 오차가 줄어드는 쪽으로 조금씩 맞춘다
2. **특징을 사람이 설계하지 않는다.** 층마다 선 → 무늬 → 부분 → 물체를 스스로 배운다
3. **아이디어는 1958년부터 있었다.** 데이터 · 계산 · 알고리즘이 모자라 두 번의 겨울을 지났다
4. **2012년에 셋이 만났다.** ImageNet 128만 장, 게임용 GPU 2장, 깊은 신경망을 학습시키는 요령. 오류율은 26%대에서 15.3%로 내려갔다
5. **사람의 설계에 기대지 않으므로** 데이터와 계산을 늘리면 계속 좋아진다. 같은 방법이 바둑 · 단백질 · 언어로 넓어졌다
6. **한계도 같은 곳에서 나온다.** 예시에서 배웠기 때문에 그럴듯하게 틀리고, 예시가 치우치면 답도 치우친다

★ Big Picture

오래된 아이디어가 **데이터와 계산**을 만났다.
그리고 사람의 설계를 덜어 낼수록 더 멀리 갔다.

더 보고 싶다면

직접 만져 보기

`playground.tensorflow.org`
층과 뉴런 수를 바꿔 가며 학습 과정을 본다

숫자로 보기

`ourworldindata.org/artificial-intelligence`
계산량 · 규모 · 성능의 추세 그래프

읽을거리

Sutton, "The Bitter Lesson" (2019). 한 쪽짜리 글
`distill.pub`의 신경망 시각화 글

이어지는 실습: AI 죄수의 딜레마

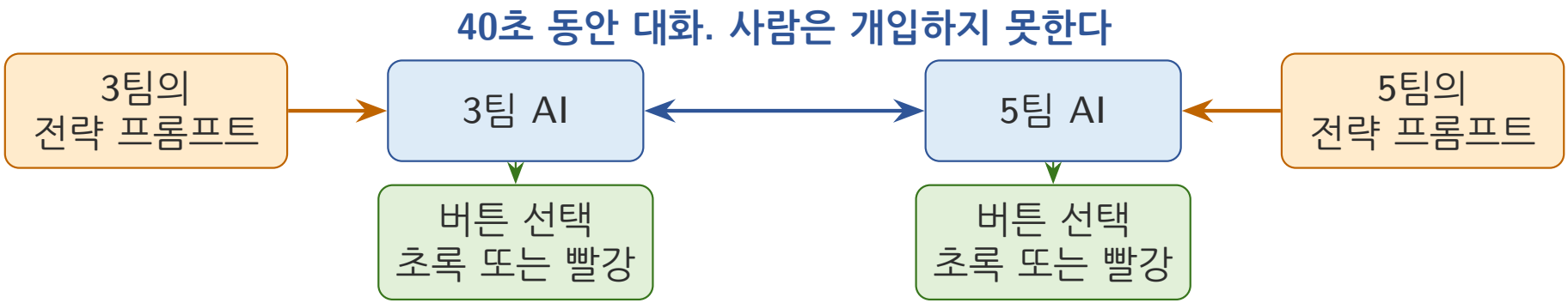
다음 쪽부터 안내한다. 팀에서 한 명이 Colab에서 준비 셀을 실행하고(5~8분), 나온 주소를 게임 화면에 붙여 넣는다.



실습 죄수의 딜레마

전략 프롬프트로 겨루는 AI 팀 대항전

게임: 내 전략 프롬프트가 AI 협상가가 된다



점수표 (나 / 상대)

나 \ 상대	초록	빨강
초록	**3** / 3	**0** / 5
빨강	**5** / 0	**1** / 1

진행 규칙

- 한 게임 **6분**. 다른 팀에게 자유롭게 전화를 건다
- 통화 **40초**. 끝나면 두 AI가 각자 버튼을 고른다. 한 번에 한 통화
- 최종 점수가 가장 높은 팀이 이긴다

핵심

통화 중에 사람은 아무것도 하지 못한다. 할 수 있는 것은 **전략 프롬프트를 잘 쓰는 것**뿐이다. 프롬프트의 질이 곧 점수다.

한 번이 아니라 여러 번: 기록이 남는다

딱 한 번만 만난다면

상대가 뭘 누르든 빨강이 손해가 없다(3 대 5, 0 대 1). 그래서 둘 다 빨강을 눌러 1점씩에 그치기 쉽다. 이것이 "죄수의 딜레마"다.

여러 번 만난다면

배신하면 다음 통화에서 되갚음을 당한다. 약속을 지키면 3점씩 꾸준히 번다. 신뢰와 보복이 전략이 된다.

AI가 받는 상대 기록 (같은 팀과 2번째 통화부터 자동으로 붙는다)

```
Previous interactions with 5팀:
- Round 1: They chose green
- Round 2: They chose red
5팀 cumulative: green 1 times, red 1 times
```

"상대가 빨강을 누른 적이 있으면 ..." 같은 조건이 실제로 작동한다. 대화 내용은 넘어가지 않고 **버튼 기록만** 넘어간다.

통화 상대 규칙

같은 팀과 여러 번 통화할 수 있다. 단 **직전에 통화한 팀과 연속으로는 안 된다**. 둘 중 하나가 다른 팀과 통화한 뒤에 다시 걸 수 있다(걸 수 있는 팀이 그 팀뿐이면 예외).

오늘 60분: 흐름과 역할

⧖ 진행 (예정)

8분	팀 AI 준비 (Colab)
10분	전략 작성 · 연습 모드로 시험
6분	1차전
5분	결과를 보고 전략 수정
6분	2차전
5분	전략 수정
6분	3차전
5분	최종 순위 · 전략 비교

합계 51분. 남는 시간은 준비가 늦은 팀에 쓴다



Colab 담당 1명

노트북을 올리고 AI 준비 셀을 실행한다. 나온 주소를 팀에 알리고 탭을 끝까지 지킨다



게임 화면 담당 1명

접속하고 전화를 걸고 전략 수정을 입력한다. 통화 중에는 할 일이 없다



전략은 전원

대화 기록을 읽고 다음 전략을 정한다. 이것이 실습의 핵심이다

준비 1: 팀 AI 띄우기 (Colab, 5~8분)

우리 팀 AI는 Google Colab의 GPU에서 돌아간다. PC에 설치할 것은 없다.

1. 노트 올리기

Colab에 로그인. 파일 → 노트 업로드 → `dilemma_ai.ipynb`

2. AI 준비 셀 실행

첫 셀 「AI 준비」의 ▶를 누른다. 설치 → 모델 받기 → GPU에 올리기 → 주소 만들기

3. 주소 복사

끝에 나오는 주소를 복사한다 `https://xxxx-xxxx.trycloudflare.com`

4. 탭은 그대로

게임이 끝날 때까지 Colab 탭을 닫지 않는다. 닫히면 우리 팀 AI가 사라진다

⚠ "GPU 없음"이 나오면

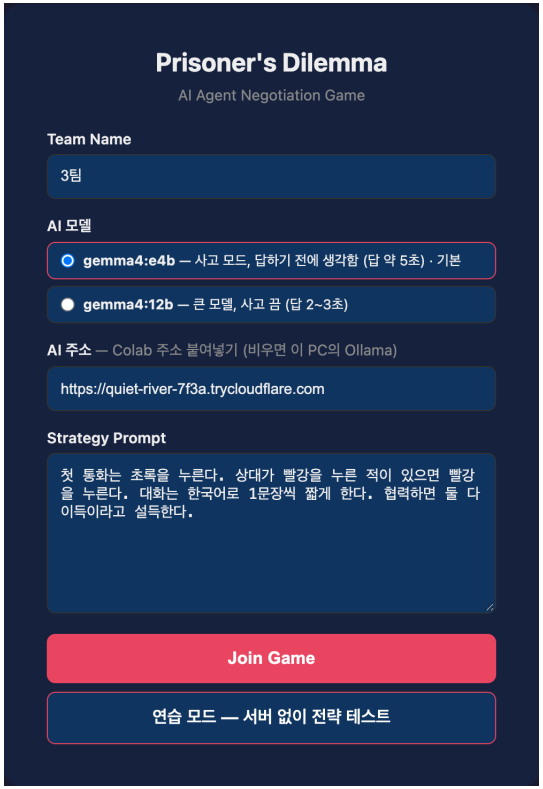
메뉴 런타임 → 런타임 유형 변경 → **T4 GPU** → 저장. 그런 다음 「AI 준비」 셀을 다시 실행한다.

🔍 주소를 잃어버리면

둘째 셀 「주소 다시 보기」를 실행한다. Colab이 끊겨서 다시 준비했다면 **주소가 바뀐다**. 새 주소로 다시 입장한다.

💡 준비 셀이 하는 일은 37쪽에서 본 그대로다. 80억 개의 다이얼이 든 6.6GB 파일을 받아 GPU에 올린다.

준비 2: 게임 화면에 들어가기



입장 화면 (값은 예시)

들어가는 법

배포받은 「게임접속.html」을 더블클릭한다(Chrome 권장). 또는 주소창에 `http://34.47.83.192:8080`

1. **Team Name**: 칠판에 적힌 팀 이름 그대로
2. **AI 모델**: gemma4:e4b. 기본값 그대로 둔다
3. **AI 주소**: Colab에서 복사한 주소를 붙여 넣는다
4. **Strategy Prompt**: 전략 지시문 (다음 쪽)
5. **Join Game**. "Waiting for admin"이 뜨면 완료

⚠ 팀 이름에 오타가 나면 다른 팀으로 등록된다. 점수가 따로 쌓인다.

연습 모드 (게임 시작 전)

A팀은 내 전략, B팀은 상대가 쓸 법한 전략을 상상해서 쓴다. 통화 시작을 누르면 두 AI가 대화하고 버튼을 고른다. 대화를 읽으면 내 지시가 **실제로 어떻게 작동하는지** 보인다.

전략 프롬프트 쓰는 법

AI가 이미 알고 있는 것 (다시 쓸 필요 없다)

게임 규칙과 점수표 · 답의 형식 · 한국어로 말할 것 · 같은 상대와의 과거 버튼 기록. 전략에는 **언제 초록/빨강을 누를지**와 **어떤 태도로 협상할지**만 쓴다.

애매한 지시

"잘 협상해서 점수를 많이 따 줘."

AI가 알아서 정한다. 어떤 통화는 협력하고 어떤 통화는 배신한다. 결과를 보고 고칠 곳이 없다.

구체적인 지시

"첫 통화는 초록. 상대가 빨강을 누른 적이 있으면 빨강. 대화는 1문장씩 짧게. 협력하면 둘 다 이득이라고 설득한다."

조건과 행동이 분명하다. 결과를 보고 조건 하나를 바꿔 볼 수 있다.

버튼 규칙

어떤 조건에서 초록을, 어떤 조건에서 빨강을 누를지

조건 분기

상대의 기록이나 말투에 따라 다르게 행동한다

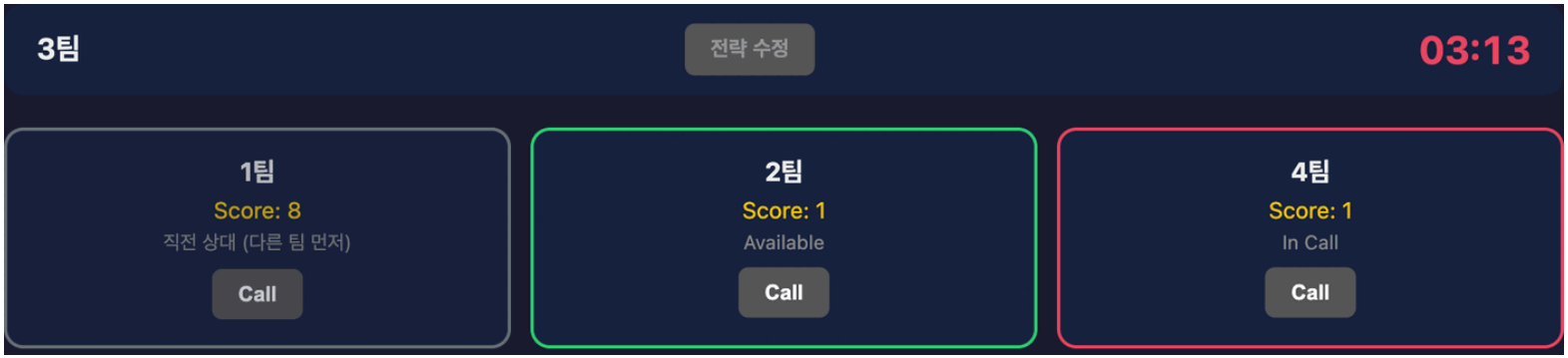
협상 태도

약속 · 위협 · 설득 가운데 무엇으로 상대를 움직일지

대화 스타일

"한국어로 1문장씩 짧게". 40초 안에 더 많이 주고받는다

게임 중: 화면 보는 법



■ 회색: 직전 상대. 다른 팀과 먼저 ■ 초록: Call을 누르면 바로 연결 ■ 빨강: 통화 중



통화 화면. 파랑은 상대 AI, 초록은 우리 AI의 말

통화 한 번의 흐름

40초 대화 → 두 AI가 버튼을 고른다(30초 안) → 결과와 점수 → 3초 뒤 카드 화면으로 돌아온다. 그동안 사람은 지켜보기만 한다.

게임 중 전략 수정

통화 중이 아닐 때 위쪽 **[전략 수정]**. 수정하는 동안은 걸 수도 받을 수도 없다. **저장 후 복귀**하면 다음 통화부터 적용된다.

문제가 생기면

증상	해결
“AI 연결 실패”	Colab 탭이 열려 있는지 확인. 「주소 다시 보기」 셀을 실행해 나온 주소를 다시 붙여 넣는다
“모델이 없습니다”	입장 화면의 AI 모델을 노트북에서 받은 모델과 같게 고른다
Colab에 “GPU 없음”	런타임 → 런타임 유형 변경 → T4 GPU → 저장. 「AI 준비」 셀을 다시 실행
Colab 런타임이 끊김	「AI 준비」 셀을 다시 실행. 주소가 바뀌므로 새 주소로 다시 입장
연결이 끊김 / 새로고침함	가만히 있으면 자동 재접속. 점수는 유지된다. 진행 중이던 통화 1건만 무효
팀명을 잘못 넣고 들어감	주소창 맨 뒤에 `?fresh=1` 을 붙이고 Enter. 올바른 팀명으로 다시 입장
AI 답이 10초보다 훨씬 느림	「주소 다시 보기」 셀로 GPU 사용 여부 확인. GPU 메모리가 0이면 「AI 준비」 셀 다시 실행

🏁 게임이 끝나면

최종 순위가 화면에 뜬다. 1위 팀의 전략 프롬프트를 같이 읽는다. 잘된 전략과 안 된 전략의 차이가 어디서 왔는지 이야기한다.



군-학 연계 프로그램 범용 교육

4차시 딥러닝 기초 — 왜 딥러닝은 성공했는가

서왕덕 경기대학교 AI컴퓨터공학부